

COMP 410: Logic Programming

Fall 2026

Instructor: Kyle Dewey (kyle.dewey@csun.edu)

Course Web Page: <https://kyledewey.github.io/comp410-fall26>

Office: JD 4419

Course Description (from the catalog)

Programming techniques in the logic programming language Prolog. Prenex conjunctive normal form and grammatical algorithms. Tableaux, sequenzen, resolution and other semi-decision procedures. Closures of relations, fixed point theory, control mechanisms, relationship to functional programming.

Learning Objectives

Successful students will be able to:

- Recognize problems which are well-suited to the logic programming paradigm
- Write Prolog programs which manipulate lists, solve NP-complete problem instances, and automatically generate software tests
- Understand how to work with logic programming features without logic programming languages, and effectively write logic programs in Python

Course Background, Emphasis, and Design

Logic programming, while often understated, is a major programming paradigm. In my opinion, logic programming is well-suited to problems with one or more of the following properties:

- There are many distinct answers of interest
- Solving the problem requires trying different approaches and seeing which works
- Answers can be described easily, but it's difficult to formulate how to arrive at an answer

This course emphasizes the use of logic programming to solve problems, along with programming techniques which are unique to the logic programming paradigm. To this end, there will be a multitude of programming assignments, most of which require you to write code. This code will be written in either Python or Prolog; Python serves as a sort of "typical" programming languages for our purposes, whereas Prolog is a classic logic programming language.

Will you ever use Prolog ever again? Honestly, probably not. However, my goal isn't to teach Prolog, but rather the ideas behind Prolog. Logic programming techniques can be implemented in more mainstream, non-logical languages; it's merely inconvenient opposed to impossible (similarly, object-oriented programming can be done in C, even though C predates object-oriented programming).

Textbook and Other Required Class Materials

No textbook is required. However, there are several sources which may be helpful:

- Learn Prolog Now! (<http://www.learnprolognow.org/lpnpag.php?pageid=online>); free online textbook that goes over the basics of Prolog
- The Art of Prolog (<https://mitpress.mit.edu/books/art-prolog>); very complete book on Prolog and logic programming which covers both basic and advanced topics; I have a copy if you'd like to peruse it
- The Craft of Prolog (<https://mitpress.mit.edu/books/craft-prolog>); discusses advanced topics in Prolog, including logic programming design patterns; I have a copy if you'd like to peruse it
- The Practice of Prolog (<https://mitpress.mit.edu/books/practice-prolog>); discusses real-world Prolog applications; I have a copy if you'd like to peruse it
- Logic, Programming, and Prolog (<https://www.ida.liu.se/~ulfni53/lpp/bok/bok.pdf>); free online textbook that focuses on the theoretical underpinnings of Prolog

Additionally, a computer, be it a laptop or otherwise, is required.

Grading

Your grade is based on the following components:

Assignments	15%
Midterm Exam 1	20%
Midterm Exam 2	30%
Final Exam	35%

Not all of the assignments will be necessarily weighted evenly, nor will you necessarily be given the same amount of time for assignments. Exactly which assignments are assigned depends on how the class progresses. In general, assignments will be submitted through Canvas (<https://canvas.csun.edu/>). In the event that there is a problem with Canvas, you may email your assignment to me (kyle.dewey@csun.edu) as a last resort.

Plus/minus grading is used, according to the scale below:

If your score is >=...	...you will receive...
92.5	A
89.5	A-
86.5	B+
82.5	B
79.5	B-
76.5	C+

If your score is >=...	...you will receive...
72.5	C
69.5	C-
66.5	D+
62.5	D
59.5	D-
0	F

If you are not present for the final exam and you have not previously made alternative arrangements with me for the final exam, a grade of WU (unauthorized withdrawal) will be assigned.

Collaboration for Assignments

All students are required to submit their own individual work. For assignments (and **only** assignments), students may discuss among each other, as long as they don't digitally share code. That is, you **cannot** simply email your code to someone else. However, you **may** discuss your actual code with someone else, including merely viewing code. The only stipulation is that **if you do discuss with someone else, say so in your submission**. This is not for punitive reasons; this is only so I get a sense of who is working with who. My intention with this policy is to enable collaborative learning, as opposed to simply sharing a solution.

Plagiarism and Academic Honesty

While collaboration is allowed on assignments, you are responsible for all of your own work. You may **not** take code from online sources and submit it as your own. If you must take code from online, cite where you took the code from. Worst-case scenario, you'll receive a 0 for whatever you took, but no further action will be taken. In general, code taken online which solves more general things (e.g., "how do I iterate through an array in Java") is more acceptable than code which solves more specific things (e.g., "how do I implement a recursive find function over immutable linked lists in Swift"). General bits of code only give you pieces of a solution, whereas specific bits of code often will give you a complete copy/pastable solution. If it's not 100% clear if something is permitted to be used or not, you can always ask me beforehand.

No discussion whatsoever is allowed during exams, except with the instructor. Any violations can result in a failing grade for the assignment/exam, or potentially failing the course for egregious cases. A report will also be made to the Dean of Academic Affairs. Students who repeatedly violate this policy across multiple courses may be suspended or even expelled.

LLM Policy

For assignments, LLM-based tools like ChatGPT are discouraged, though not outright disallowed. It can be easy to overly on such tools without realizing it, to the point where the tool does all the work, with zero understanding of what it produces. If you do use such tools, you should be aware of the following trap I've seen a lot of students fall into over the past year:

- The student gets a solution from an LLM
- The student tests the solution, and observes that all tests pass
- The student looks at the code a bit, and decides that it "feels" right. This feeling is usually based on the relative length, complexity, formatting, and variable naming of the code, and not on its actual behavior.
- The student bombs a high-scoring, closely-related exam question, which asked them to write similar code for a similar problem.
- The student realizes they did not actually understand what the LLM produced, and then needs additional help to try to catch up.

Keep in mind that LLMs fundamentally just generate some text that statistically is likely to follow whatever the prompt is based on the training set, with no understanding of what the text says. Any output from an LLM **must** therefore be verified to ensure it is correct. Since we are learning new topics, you will unlikely be able to verify by looking at the code alone. They may still be useful for pointing you in some direction if you are lost, but there is no guarantee that the direction is even relevant (and I've helped students out of a completely irrelevant hole more than once).

In general, for this course, my opinion is that if an LLM is to be used, it's best to use prompts that are general (e.g., how does recursion in Prolog work?) as opposed to specific (e.g., write a Prolog program using recursion that solves the specific problem I've been asked to solve). A course-specific pitfall is that there is far less Prolog in LLM training sets than for more popular languages, and so LLMs have a tendency to produce incorrect code, especially for atypical problems.

Communication

In general, any questions should be made through Canvas. You can also email me, though I'm usually much faster to respond to Canvas than my general email.

Late Policy / Exam Scheduling

Late assignments will be accepted without penalty if prior arrangements have been made or there is some sort of legitimate emergency (at my discretion). If you must be absent from an exam, contact me ASAP to see if alternative accommodations can be made.

If an assignment is otherwise submitted late, it will be penalized according to the following scale:

If your assignment is late by $\leq$ this many days...	...it will be deducted by...
1	10%
2	30%
3	60%
4+	100%

To be clear, assignments which are submitted four or more days beyond the deadline will not receive credit. The reason for such a harsh late policy is that we will generally discuss solutions in class shortly after the deadline, and this late policy discourages people from simply pulling a solution from an in-class discussion.

Class Feedback

I am open to any questions / comments / concerns / complaints you have about the class. If there is something relevant you want covered, I can push to make this happen. I operate off of your feedback, and no feedback tells me “everything is ok”.

Class Schedule and List of Topics (Subject to Change)

Week	Monday	Wednesday
1	8/24: Introduction, motivation	8/26: SAT, semantic tableau
2	8/31: SAT, semantic tableau	9/2: SAT, semantic tableau, abstract syntax trees
3	9/7: Labor day (no class)	9/9: Abstract syntax trees, introduction to Prolog, nondeterminism, unification
4	9/14: Nondeterminism, unification	9/16: Unification with structures, recursion
5	9/21: Unification with structures, recursion	9/23: Unification with lists
6	9/28: Unification with lists, recursion with lists	9/30: Unification with lists, recursion with lists
7	10/5: Midterm exam 1 review	10/7: Midterm Exam 1
8	10/12: Midterm exam 1 retrospective, generating tests	10/14: Context-free grammars, generating tests

Week	Monday	Wednesday
9	10/19: Generating tests	10/21: Generating tests
10	10/26: Generating tests, optimizing Prolog code	10/28: Generating tests, optimizing Prolog code, Prolog control structures
11	11/2: Prolog control structures	11/4: Midterm Exam 2 Review
12	11/9: Midterm Exam 2	11/11: Veterans Day (no instruction)
13	11/16: Nondeterminism via iterators, nondeterminism in Python	11/18: Nondeterminism in Python
14	11/23: Nondeterminism in Python	11/25: Nondeterminism via iterators More nondeterminism in Python, implementing unification in non-logical languages
15	11/30: Nondeterminism via iterators	12/2: Implementing unification in non-logical languages
16	12/7: Combined unification and nondeterminism in non-logical languages	12/9: Final Exam Review

Final exam: Monday, December 14 from 3 - 5 PM in JD 1600B